This is a fantastic next step! Simulating the entire vibration-sensing and decision-making pipeline in MATLAB will save you countless hours of debugging once you move to the physical ESP32 and MPU6050 hardware. 🛠️

Here is your step-by-step explanatory guide to building the MATLAB simulation for the seismic detection module. This simulation will model the raw MPU6050 data, filter out background noise, apply the Fast Fourier Transform (FFT) to isolate the frequency footprint, and trigger a simulated LoRa transmission. 🐘 🔗

Step 1: Simulate the Raw MPU6050 Sensor Data

Since we do not have live hardware hooked up to an elephant's path right now, we will mathematically synthesize the vibration data. Elephants generate low-frequency (10-50Hz) high-amplitude impulses. Humans or vehicles generate higher frequency, irregular impulses. 🔗

We will create a raw signal that combines:

- A **20 Hz** heavy impact (simulating the elephant).
- An **80 Hz** high-frequency vibration (simulating a passing vehicle or wind).
- Random white noise.

Step 2: Apply a Digital Low-Pass Filter

A raw vibration signal is too messy for an AI or logic gate to read directly. We need a hardware or software filter to cut off all sounds above 50Hz (wind, birds, cars). We will design a 4th-order Butterworth Low-Pass Filter in MATLAB with a 50Hz cutoff frequency. This smooths the data and isolates the heavy footsteps. 🔗

Step 3: Perform the Fast Fourier Transform (FFT)

Time-domain data (amplitude over time) is hard to classify. By applying an FFT algorithm, we translate the signal from the time domain to the frequency domain to isolate the frequency footprint. This will give us a clear "spike" at the exact frequencies present in the ground. 🔗

Step 4: The Classification & LoRa Trigger Logic

We will write a simple decision logic: If the most prominent frequency (the peak of the FFT) falls between 10 Hz and 50 Hz, and its amplitude is high enough, the system classifies it as a threat.

Once classified as a threat, the code will output a simulated command to the Ra-02 LoRa module, which provides long-range (10km+) communication. 🔗

The Complete MATLAB Simulation Code 💻

You can copy and paste this entire block into a new MATLAB script (e.g., `GajaShield_Sim.m`) and run it.

Matlab

```matlab
% ---------------------------------------------------------
% Pathfinder / GajaShield: MPU6050 & LoRa Simulation
% ---------------------------------------------------------

%% 1. Generate Synthetic MPU6050 Vibration Data
fs = 200;                 % Sampling frequency (200 Hz is enough for <100Hz
signals)
t = 0:1/fs:5;             % Simulate 5 seconds of data

% Create the "Elephant" footprint (20 Hz, High Amplitude)
elephant_impact = 3.0 * sin(2*pi*20*t);

% Create "Background Noise" (80 Hz vehicle + random noise)
vehicle_noise = 1.5 * sin(2*pi*80*t);
random_noise = 0.5 * randn(size(t));

% Combine to form the raw sensor reading (Z-axis vibration)
raw_z_axis = elephant_impact + vehicle_noise + random_noise;

%% 2. Apply Low-Pass Filter (Cutoff at 50 Hz)
fc = 50;                  % Cutoff frequency of 50 Hz
% Calculate normalized cutoff frequency for the digital filter
Wn = fc / (fs/2);
% Design a 4th-order Butterworth low-pass filter
[b, a] = butter(4, Wn, 'low');

% Apply the filter to our raw data (zero-phase filtering)
filtered_z = filtfilt(b, a, raw_z_axis);

%% 3. Apply FFT (Fast Fourier Transform)
L = length(filtered_z);
% Compute the FFT of the filtered signal
Y = fft(filtered_z);
% Calculate the two-sided spectrum and then the single-sided spectrum
P2 = abs(Y/L);
P1 = P2(1:floor(L/2)+1);
P1(2:end-1) = 2*P1(2:end-1);
% Define the frequency domain axis
f = fs*(0:(L/2))/L;

%% 4. Classification & LoRa Transmission Logic
% Find the dominant frequency (highest amplitude peak)
[max_amplitude, max_index] = max(P1);
dominant_freq = f(max_index);

fprintf('--- ANALYSIS RESULTS ---\n');
fprintf('Dominant Frequency Detected: %.2f Hz\n', dominant_freq);
fprintf('Peak Amplitude: %.2f\n', max_amplitude);

% Decision Gate: Is it an elephant? (10-50Hz range with high amplitude)
amplitude_threshold = 1.5;

if (dominant_freq >= 10 && dominant_freq <= 50) && (max_amplitude >
amplitude_threshold)
    disp('CLASSIFICATION: Large Animal / Elephant Detected! 🐘');
    disp('TRIGGERING LORA TX: "NODE_01, ALERT: HEC, DIST: <500M" 📡');
else
    disp('CLASSIFICATION: Normal Background Noise.');
    disp('LORA TX: "NODE_01, STATUS: SAFE"');
end

%% 5. Plotting the Results for Visualization
figure;

% Plot 1: Raw vs Filtered Data
subplot(2,1,1);
plot(t, raw_z_axis, 'Color', [0.8 0.8 0.8]); hold on;
plot(t, filtered_z, 'b', 'LineWidth', 1.5);
title('Time Domain: Raw MPU6050 Data vs 50Hz Low-Pass Filtered Data');
xlabel('Time (seconds)');
ylabel('Amplitude (g)');
legend('Raw Noisy Signal', 'Filtered Signal');
grid on;

% Plot 2: Frequency Spectrum (FFT)
subplot(2,1,2);
plot(f, P1, 'r', 'LineWidth', 1.5);
title('Frequency Domain: FFT Spectrum of Filtered Signal');
xlabel('Frequency (Hz)');
ylabel('Amplitude');
xlim([0 100]); % Only show up to 100 Hz
grid on;
```

Would you like me to outline how to convert this MATLAB logic into C++ so you can flash it onto your NodeMCU to read the real MPU6050 sensor?